Documentation de Validation

GL03



CAMATCHY Alexander
CANAGUY Vincent
GRAHAM Martha
GUINET Aster
LAISNÉ Matthew
MATER Hugo
NGONGANG Gaston
THOMAZIC Valentin
WENDELS Jérémy

Table des matières

1. Descriptif des tests _____ -3-
2. Scripts de tests _____ -5-
3. Gestion des risques et gestion des rendus _____ -6-
4. Gestion des mises en production _____ -11-
5. Résultats de Jacoco _____ -12-
6. Autres méthodes de validation _____ -13-
   6.1. Relecture du code des autres _____ -13-
   6.2. Pipelines sur GitLab _____ -13-
   6.3. Utilisation de git _____ -13-

1. Descriptif des tests

Tests du compilateur Decac

Les tests sont organisés en 3 répertoires « syntax », « context » et « codegen », correspondant aux différentes passes du compilateur.

Chaque répertoire contient un répertoire « valid », contenant les tests devant passer sans lever d'erreur. Les répertoires « syntax » et « context » possèdent un répertoire « invalid », dont les tests contiennent des programmes qui doivent être refusés par le compilateur.

Convention de nommage des tests

Pour un incrément donné `monIncrement`, chaque répertoire « valid » ou « invalid » peut contenir un répertoire de la forme `<id>.monIncrement`, où l'identifiant est unique et propre à l'incrément. Pour chaque incrément on trouve des fichiers de la forme

- `<i><feature>.deca`
- `<i><feature>.expected`

Si plusieurs fichiers concernent une même fonctionnalité, on les regroupe sous un répertoire.

Chaque nom de fichier ou fonctionnalité est de préférence en camelCase.

Chaque identifiant est unique pour chaque répertoire, et permet d'organiser les tests.

Fichiers requis pour un test monTest dans « syntax » et « context »

`valid` : uniquement un fichier `<id>monTest.deca`

`invalid` :

- un fichier `<id>monTest.deca`
- un fichier `<id>monTest.expected`

Les fichiers « expected » doivent contenir l'endroit où l'erreur sera détectée au format `<nom_fichier>:<ligne>`.

Fichiers requis pour un test `monTest` dans `codegen/valid` :

- un fichier `<id>monTest.deca`
- un fichier `<id>monTest.expected` par `<id>monTest.deca`
- un fichier `<id>monTest.input` facultatif contenant les entrées à transmettre aux fonctions d'entrée dans le programme (en cas de readInt par exemple). Une donnée par ligne, dans l'ordre chronologique des appels.

Exemples

Fichiers et dossiers de tests de « context »: « invalid » et « valid ».

```
src/test/deca/context/invalid/01.renduInterm1/00expr
├── 00equals
│   ├── 00equalsBoolInt.deca
│   ├── 00equalsBoolInt.expected
│   ├── 01equalsBoolFloat.deca
│   [...]
├── 01leq
│   ├── 00leqBool.deca
│   ├── 00leqBool.expected
│   ├── 01leqBoolInt.deca
│   [...]
├── 02less
[...]

src/test/deca/context/valid/01.renduInterm1/00expr
├── 00equals.deca
├── 01leq.deca
├── 02less.deca
├── 03or.deca
├── 04and.deca
└── 05not.deca
```

Fichiers et dossiers de tests de « codegen ».

```
src/test/deca/codegen/valid/01.renduInterm1/
├── 00devinetteDicho.deca
├── 00devinetteDicho.expected
├── 00devinetteDicho.input
├── 00expr
│   ├── 00equals.deca
│   ├── 00equals.expected
│   ├── 01leq.deca
│   [...]
├── 01inst
│   ├── 00while
│   │   ├── 00while.deca
│   │   └── 00while.expected
│   └── 01ifthenelse
│       ├── 00if.deca
│       [...]
├── 02primaryExpr
│   ├── 00readInt.deca
│   ├── 00readInt.expected
│   └── 00readInt.input
[...]
```

2. Scripts de tests

Pour chaque incrément, on prévoit un script de test :

Incrément Initial : `renduInitial.sh`

Incrément 1 : `renduInterm1.sh`

Déca sans objet : `renduInterm2.sh`

RenduFinal : `renduFinal1.sh`

Qui utilisent des scripts de test plus généraux :

`test_syntax_repo.sh [rep] [-r]`: exécute test_syntax sur chacun des tests présent récursivement dans `syntax/(valid|invalid)/[rep]/`.

`test_context_repo.sh [rep] [-r]`: exécute test_context sur chacun des tests présent récursivement dans `context/(valid|invalid)/[rep]/`.

`test_codegen_repo.sh [rep] [-r]`: exécute test_syntax sur chacun des tests présent récursivement dans `codegen/valid/[rep]/`.

L'option `-r` permet de choisir s'il faut recompiler le projet à chaque fois.

Le script `summary.sh` affiche le résultat global de tous les tests.

Le script `clean.sh` permet de supprimer tous les fichiers générés par les tests, d'extension `res`, `ass` et `debug`.

Le script `clean_tmp.sh` permet de supprimer tous les fichiers temporaires (générés pour `summary.sh`).

3. Gestion des risques et gestion des rendus

Risques techniques

Numéro	Nom	Description	Prévention	Réparation
T1	Régression	En avançant sur les incréments les tests précédents ne passent plus	Mettre en place une CI, effectuer des tests de non-régression	Utiliser l'historique et les branches Git. (Nécessite une bonne utilisation de Git)
T2	Crash des plateformes (Jira, GitLab)	Impossibilité d'accéder au Gitlab Ensimag ou à Jira, voire une perte des données du projet hébergées	Mise en place d'un miroir Gitlab sur Gitlab.com	Récupération des données sur le miroir Gitlab
T3	Manque de compétences	Pour un ou plusieurs membres, manque de compétences dans un des domaines nécessaires au projet	Bonne répartition des tâches en fonction des forces de chacun	Pair programming afin d'aider à la montée en compétences de tous
T4	Manque de bonnes pratiques de code	Utilisation de styles de code différents, noms de variables non explicitemppms…	Formateur de code commun pour tous, lecture du code des autres	Dernier recours : réécriture du code

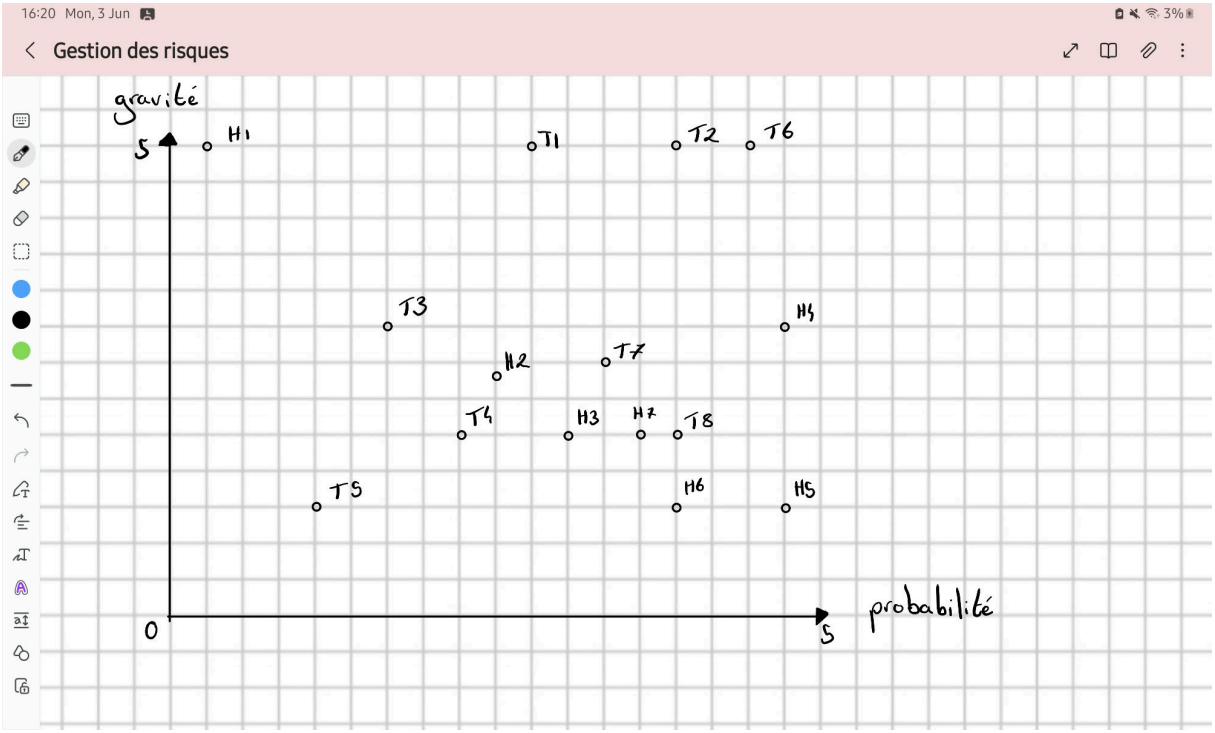
Numéro	Nom	Description	Prévention	Réparation
T5	Incompatibilité des technologies	En fonction des PC de chacun, des système d'exploitation, des éditeurs de code, il peut y avoir des problèmes de compatibilité	Essayer d'utiliser le même éditeur de code, même formateur etc	Changer de technologie ou demander de l'aide à ceux qui ont compris
T6	Non respect du cahier des charges	Mauvaise fonctionnalité développée	Bien lire la doc et se tenir au courant de l'avancement pour détecter rapidement des problèmes	Dernier recours : réécriture du code
T7	Manque de tests	Oubli de tester un incrément, une fonction…	Formulation des tests avant le code	Ecriture de plus de tests par d'autres
T8	Manque de documentation	Absence de commentaire pour chaque fonction, code non explicite	Chaque personne qui code se charge de commenter son propre code	Faire des points sur les parties incomprises et les documenter ensemble pour que tous les comprennent

Risques humains

Numéro	Nom	Description	Prévention	Réparation
H1	Départ d'un membre de l'équipe	Un membre de l'équipe abandonne la formation, a des problèmes personnels…	Discussion avec le respo risque humain	Redistribution des tâches
H2	Personne malade	Un membre de l'équipe tombe malade		Redistribution des tâches ou travail ou télétravail si possibilité
H3	Temps de réponse longs	Les membres de l'équipe ne se rendent pas disponibles pendant les horaires de travail	S'accorder sur des horaires de travail obligatoires	Se ré-accorder sur les horaires de travail
H4	Burn out / Perte de motivation	Burn out	Discussions avec le respo risque humain	Redistribution des tâches
H5	Manque de ponctualité	Arrive en retard ou ne vient pas	Petits déj au daily pour motiver	
H6	Perte d'informations	Problème de communication au sein de l'équipe menant à des actions potentiellement inutiles, ou	Daily meeting	Discussion sur le sujet de manière plus approfondie au daily meeting

Numéro	Nom	Description	Prévention	Réparation
		à un manque d'actions utiles		
H7	Conflits personnels		Temps de cohésion pendant le daily meeting	Discussion avec le médiateur

Graphe probabilité x gravité des risques



4. Gestion des mises en production

Nous avons établi une checklist d'actions à effectuer avant chaque rendu / version de notre compilateur :

Action	**Explication**	**Raison**
Vérifier les tickets Jira	Vérifier que tous les tickets ont été marqués comme **DONE**	Pour que le burn-down chart soit le plus réaliste possible, et vérifier que nous avons pas oublié une fonctionnalité
Vérifier que toutes les Merge Requests ont été validé	Aller sur GitLab vérifier toutes les branches	Empêcher les oublis de features
Relecture de la documentation par quelqu'un qui ne l'a pas écrite	Vérifier les fautes d'orthographe et que rien ne soit oublié dans la doc	On ne voit que difficilement ses propres erreurs
Vérifier que les tests passent	On re télécharge le code depuis notre gitlab et on relance tous les tests	Vérifier qu'il n'y a eu aucun oubli de push

5. Résultats de Jacoco

À chaque commit poussé sur les branches `develop` et `master` du dépôt Git, un rapport de couverture est généré et publié à l'adresse suivante : https://glapp2024.pages.ensimag.fr/gl03/index.html.

En l'état, les résultats ne sont pas exploitables mais l'infrastructure est mise en place. L'idée de la validation avec Jacoco que nous comptons mettre en place est d'exécuter des tests unitaires sur les classes ajoutées au projet comme les Manager(Error, Stack, Label…), et de lancer nos programmes de tests avec Jacoco. Cela permettra de mesurer dans quelle mesure nos tests Deca couvrent bien tous les cas prévus par notre compilateur, qu'il s'agisse de la vérification ou de la génération de code.

Capture d'écran de la couverture actuelle avec Jacoco

Deca Compiler > fr.ensimag.deca.codegen

fr.ensimag.deca.codegen

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
LabelManager		38%		25%	19	24	36	59	11	16	0	1
ErrorManager		0%		0%	9	9	22	22	6	6	1	1
StackManager		0%		0%	11	11	22	22	10	10	1	1
GenericError		0%		n/a	3	3	10	10	3	3	1	1
LabelBoolAssign		0%		n/a	4	4	5	5	4	4	1	1
LabelWhile		0%		n/a	3	3	4	4	3	3	1	1
LabelIf		0%		n/a	3	3	4	4	3	3	1	1
IOError		0%		n/a	2	2	7	7	2	2	1	1
TSTOError		0%		n/a	2	2	7	7	2	2	1	1
DivZeroError		0%		n/a	2	2	7	7	2	2	1	1
AbstractLabel		62%		n/a	2	4	3	10	2	4	0	1
LabelOr		0%		n/a	2	2	3	3	2	2	1	1
LabelBoolExp		0%		n/a	2	2	3	3	2	2	1	1
AbstractError		0%		n/a	1	1	2	2	1	1	1	1
LabelAnd		100%		n/a	0	2	0	3	0	2	0	1
Total	563 of 697	19%	20 of 24	16%	65	74	135	168	53	62	12	15

6. Autres méthodes de validation

6.1. Relecture du code des autres

Afin de s'assurer de la qualité du code, chaque ticket correspond à une branche. A la fin du développement, une merge request est créée et celle-ci doit être relue par au moins un développeur, un testeur et un code owner.

6.2. Pipelines sur GitLab

Afin de s'assurer de la non-régression lors du développement du projet, un pipeline sur gitlab lance tous les script de test sur master et develop à chaque push. De plus la pipeline lance également les tests une fois par nuit sur master et develop afin que l'on puisse arriver le matin et être sûr que tout fonctionne.

6.3. Utilisation de git

Pour que notre projet soit le plus possible qualitatif, nous avons mis en place beaucoup de règle sur git et avons également tenu une documentation pour faciliter son utilisation par l'équipe tout au long du projet. Ce document se trouve au même endroit que les 3 documents à rendre. (docs/)